



THE UNIVERSITY OF
SYDNEY

Student number:

Room number:

Seat number:

ANONYMOUSLY MARKED

(Please do not write your name on this exam paper)

CONFIDENTIAL EXAM PAPER

This paper is not to be removed from the exam venue.

Computer Science

EXAMINATION

Semester 2 – Main, 2025

COMP2123 Data Structures and Algorithms

EXAM WRITING TIME: 120 minutes**READING TIME:** 10 minutes**EXAM CONDITIONS:**

This is a RESTRICTED OPEN book exam - specified materials permitted.

MATERIALS PERMITTED IN THE EXAM VENUE:

No electronic aids are permitted (e.g., laptops, phones)

Bilingual dictionary, one A4 sheet of handwritten and/or typed notes double-sided.

MATERIALS TO BE SUPPLIED TO STUDENTS:

This exam paper booklet.

INSTRUCTIONS TO STUDENTS:

DO NOT ATTACH EXTRA PAGES TO THIS EXAM BOOKLET. **Neatly** write your student ID in the box at the top right of this front page. Write your answers in the spaces provided in the exam paper booklet using blue or black ink. If you run out of space in the space provided right after each problem statement, continue your answers on pages 19-23.

Please tick this box to confirm that your examination paper is complete ☐**For examiner use only:**

Problem	1	2	3	4	5	Total
Marks						
Out of	20	20	20	20	20	100

Problem 1. Each question has **only one** correct answer: You will be awarded a score +2 for selecting only the correct answer (like so ☐) , and 0 otherwise.

(1) Consider the following function:

```
1: def HARDER_MYSTERY(n)
2:     i = 2
3:     while i < n do
4:         j = 1
5:         while j < i do
6:             Print j
7:             j = j * 2
8:         i = i * 2
```

What is the tightest upper bound on the time complexity of this function?

- ☐ $O(\log n)$
- ☐ $O(\log^2 n)$
- ☐ $O(n^2)$
- ☐ None of the above

(2) You are given the following sequence of operations on an initially empty stack and queue:

1. Push 1 onto the stack
2. Enqueue 2 into the queue
3. Push 3 onto the stack
4. Enqueue 4 into the queue
5. Pop from the stack and enqueue the result into the queue
6. Dequeue from the queue and push the result onto the stack

What are the final contents of the stack (top to bottom) and the queue (front to rear)?

- ☐ Stack: [2, 1], Queue: [4, 3]
- ☐ Stack: [1, 2], Queue: [4, 3]
- ☐ Stack: [2, 1], Queue: [3, 4]
- ☐ Stack: [1, 2], Queue: [3, 4]

- (3) You are given the following three traversal sequences of a binary tree:

Inorder: D B E A F C

Postorder: D E B F C A

Which of the following is the Preorder traversal of the same tree?

- ☐ A B D E C F
- ☐ A C F B E D
- ☐ A B C D E F
- ☐ None of the above

- (4) Let T be a binary search tree containing n distinct integers. You are given two integers a and b such that $a < b$. Which of the following statements is always true about the number of nodes in T whose values lie in the inclusive range $[a, b]$?

- ☐ It is at most $\log n$
- ☐ It is at least $b - a$
- ☐ It is at most n
- ☐ It is exactly $b - a + 1$

- (5) You are given an unsorted array of n elements. You want to sort it using a priority queue implemented as a binary heap. Which of the following statements is true about the time complexity of this approach?

- ☐ Building the heap takes $O(n \log n)$ time, and extracting all elements takes $O(n \log n)$, so total time is $O(n \log n)$.
- ☐ Building the heap takes $O(\log n)$ time, and extracting all elements takes $O(n \log n)$, so total time is $O(n \log n)$.
- ☐ Both building the heap and extracting elements take $O(n)$ time, so total time is $O(n)$.
- ☐ Building the heap takes $O(n \log n)$, and extracting elements takes $O(n)$, so total time is $O(n \log n)$.

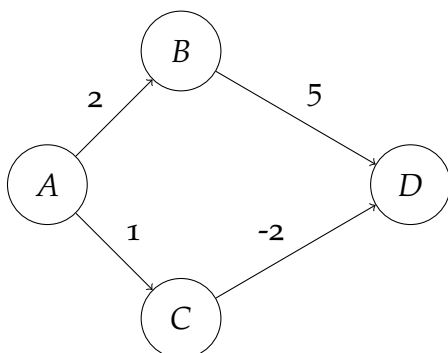
- (6) In a hash table using linear probing for collision resolution, which of the following operations is most likely to degrade in performance as the load factor approaches 1, even if the hash function distributes keys uniformly at random (i.e., each key is equally likely to hash to any table position)?

- ☐ Insertion
- ☐ Search for an existing key
- ☐ Search for a non-existing key
- ☐ All of the above

- (7) You are given an unweighted, connected graph representing cities and roads. You want to find the shortest path (i.e., path with fewest edges) from City A to City B. Which of the following statements is correct?

- ☐ DFS is guaranteed to find the shortest path if all nodes are visited.
- ☐ BFS is guaranteed to find the shortest path because it explores nodes in increasing distance from the source.
- ☐ DFS is faster than BFS and therefore better for shortest path problems.
- ☐ BFS may miss the shortest path if the graph contains cycles.

- (8) You run Dijkstra's algorithm on a graph with the following edges:



What distance will Dijkstra's algorithm compute from A to D?

- ☐ 3
 - ☐ 4
 - ☐ 6
 - ☐ It may compute an incorrect result
- (9) You are given the following characters and their frequencies:
- A : 5
 - B : 9
 - C : 12
 - D : 13
 - E : 16
 - F : 45

Using Huffman's algorithm, what is the total number of bits required to encode the string "ABCDEF" once?

- ☐ 18
- ☐ 14
- ☐ 16
- ☐ None of the above

(10) If you apply Merge Sort (divide-and-conquer) to an array of size n , what is the depth of the recursion until you reach subarrays of size 1?

☐ $\log n$

☐ n

☐ $n \log n$

☐ $\frac{n}{2}$

Problem 2. Given an array of integers $A = [a_1, a_2, \dots, a_n]$ with $n \geq 1$, find the contiguous subarray (containing at least one element) that has the largest sum. Return the sum of that subarray. [20 marks]

Example:

$$A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$$

The contiguous subarray $[4, -1, 2, 1]$ has sum 6, which is the maximum possible. Thus, the output is 6.

Your **task** is to answer the following:

Consider the following faulty algorithm:

```

1: def FACULTY_MaxSUBARRAY(A)
2:    $i = 0, sum = 2, max\_sum = -\infty$ 
3:   while  $i < \text{sizeof}(A)$  do
4:      $sum = sum + A[i]$ 
5:     if  $sum < 0$  then
6:        $sum = 0$ 
7:     if  $sum > max\_sum$  then
8:        $max\_sum = sum$ 
9:      $i = i + 1$ 
   return  $max\_sum$ 

```

Explain in one sentence why **Faulty_MaxSubarray(A)** fails. Provide a counterexample input array where this algorithm gives an incorrect result. State both the expected correct answer and the output produced by the algorithm for your counterexample.

Problem 3. Let $G = (V, E)$ be a simple weighted undirected graph consisting of n vertices and m edges, given in adjacency list representation. All edge weights are non-negative integers. [20 marks]

You are also given two fixed and distinct nodes:

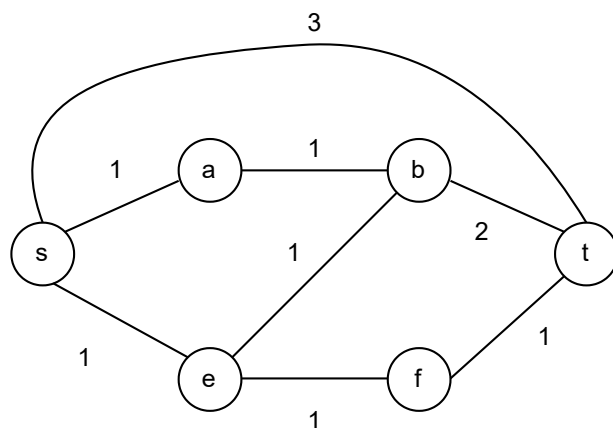
$$(u_1, v_1) \in V.$$

You are also given two distinct source and target vertices $s, t \in V$. Your **task** is to design an algorithm that computes the length of the shortest path from s to t that satisfies the following constraint:

Constraint: The path must include *both* nodes u_1 and v_1 or include *neither*. Paths that include only one of these two nodes are invalid and must be discarded. The order between u_1 and v_2 in your path does *not* matter, i.e., u_1 can appear before v_1 or after v_1 .

The path you return does not have to be simple (vertices may repeat), but it must satisfy the constraint exactly. If no such path exists, you may return ∞ .

Example:



Let the two disjoint nodes be: $u_1 = a, v_1 = e$.

Valid paths:

- $s \rightarrow t$: total weight 3.
- $s \rightarrow a \rightarrow b \rightarrow e \rightarrow f \rightarrow t$: total weight $1 + 1 + 1 + 1 + 1 = 5$.

Invalid paths:

- $s \rightarrow a \rightarrow b \rightarrow t$: includes only u_1 .
- $s \rightarrow e \rightarrow f \rightarrow t$: includes only v_1 .

Shortest valid path: Among all valid paths, the path $s \rightarrow t$ has the minimum weight of 3. Hence, the algorithm should return 3.

Your **task** is to give an algorithm for this problem that runs in $O(m + n \log n)$ time.

a) Describe your algorithm in plain English.

b) Analyze the running time of your algorithm.

c) Prove the correctness of your algorithm.

Problem 4. You are given an array A containing n words. You are asked to design a [20 marks]
data structure that supports the following query: given a string q , return the number of
words in A that either start with q or end with q . For simplicity, assume all words in A
are distinct.

Example: Let $A = ["apple", "app", "banana", "band", "bandana", "cat"]$

- If we query with $q = "app"$, the result should be 2 ("apple", "app").
- If we query with $q = "ana"$, the result should be 2 ("banana", "bandana").
- If we query with $q = "ppl"$, the result should be 0.

Your **task** is to design a data structure for the above problem that:

- Uses $O(n \cdot L)$ space, where L is the maximum size of a word that is independent of n .
- Can be constructed in $O(nL \log n)$ time.
- Supports each query in $O(|q| \log n)$ time.

You **do not** need to support insertion or deletion of new words.

a) Describe your data structure implementation in plain English.

b) Analyze the time and space complexity of your data structure.

c) Prove the correctness of your data structure.

Problem 5. Imagine you are given a set of n points scattered on a flat surface — think of them as the positions of cities on a map, or stars in a 2D galaxy projection. We want to find the two points that are closest to each other. [20 marks]

Distance measure. The notion of “closeness” is based on **Euclidean distance**, the standard straight-line distance from elementary geometry. If one point is at (x_1, y_1) and another at (x_2, y_2) , then the distance is

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

This is just an application of the Pythagorean theorem: the horizontal difference is $|x_1 - x_2|$, the vertical difference is $|y_1 - y_2|$, and the diagonal (hypotenuse) is the actual distance.

Problem statement: Given a set of n points in 2D, find the pair of points (p_i, p_j) with the minimum Euclidean distance.

Example:

Consider the points

$$P = \{(0,0), (3,4), (1,1), (5,0), (2,2)\}.$$

- Distance between $(0,0)$ and $(1,1)$ is $\sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2}$.
- Distance between $(1,1)$ and $(2,2)$ is also $\sqrt{2}$.
- All other pairs are farther apart.

So the closest pair is either $((0,0), (1,1))$ or $((1,1), (2,2))$, with distance $\sqrt{2}$. One of these two pairs must be returned.

Your **task** is to provide a divide and conquer algorithm with a running time that is polynomial in n that solves this problem.

a) Describe your algorithm in plain English.

b) Prove the correctness of your algorithm.

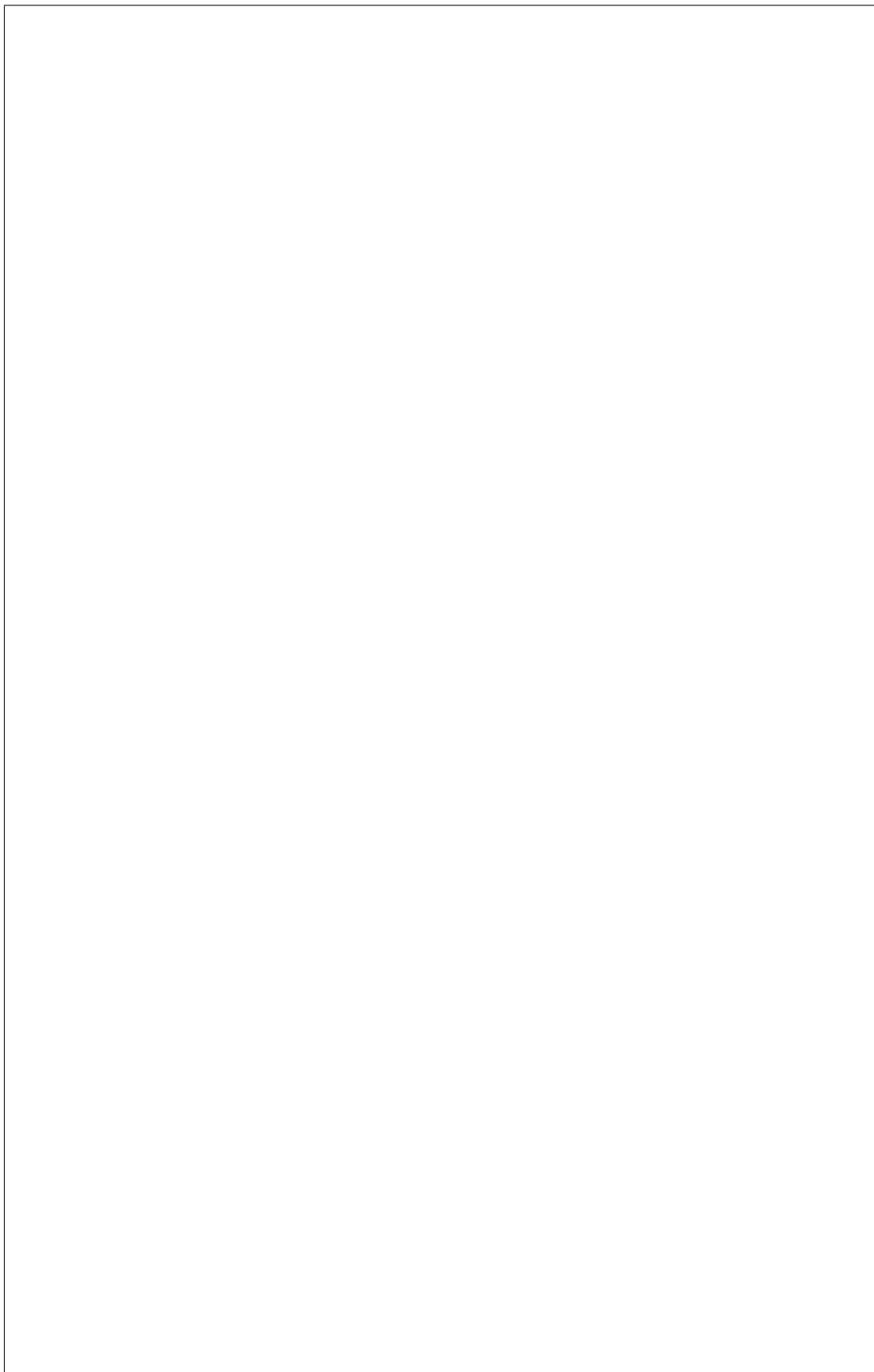
c) Analyze the running time of your algorithm.

THIS AND FOLLOWING PAGES LEFT INTENTIONALLY BLANK.

Use this and following pages for additional writing space if you run out of space in an answer area. If you want any writing on this page or following pages to be marked, you **MUST** write “see additional pages” in the relevant answer area and clearly indicate the question number you are answering on the relevant page. Any writing outside of this booklet will not be examined.



(use the rest of the space of rough work)

A large, empty rectangular box with a thin black border, occupying the majority of the page below the instruction. It is intended for students to use for rough work during the exam.

(use the rest of the space of rough work)

A large, empty rectangular box with a thin black border, occupying the majority of the page below the instruction. It is intended for students to use for rough work during the exam.

(use the rest of the space of rough work)

A large, empty rectangular box with a thin black border, occupying the majority of the page below the instruction. It is intended for students to use for rough work during the exam.

(use the rest of the space of rough work)

A large, empty rectangular box with a thin black border, occupying the majority of the page below the header and above the footer. It is intended for rough work or calculations.